```
  class OrderMailer < ApplicationMailer
»   default from: "Sam Ruby <depot@example.com>"
    # Subject can be set in your I18n file at config/locales/en.yml
    # with the following lookup:
    #
    #   en.order_mailer.received.subject
    #
    def received
      @greeting = "Hi"

      mail to: "to@example.org"
    end

    # Subject can be set in your I18n file at config/locales/en.yml
    # with the following lookup:
    #
    #   en.order_mailer.shipped.subject
    #
    def shipped
      @greeting = "Hi"

      mail to: "to@example.org"
    end
  end
```

If you're thinking to yourself that this looks like a controller, that's because it does. It includes one method per action. Instead of a call to `render`, there's a call to `mail`. This method accepts a number of parameters including `:to` (as shown), `:cc`, `:from`, and `:subject`, each of which does pretty much what you'd expect it to do. Values that are common to all `mail` calls in the mailer can be set as defaults by simply calling `default`, as is done for `:from` at the top of this class. Feel free to tailor this to your needs.

The comments in this class also indicate that subject lines are already enabled for translation, a subject we'll cover in Chapter 15, *Task J: Internationalization*. For now, we'll simply use the `:subject` parameter.

As with controllers, templates contain the text to be sent, and controllers and mailers can provide values to be inserted into those templates via instance